

1. feladat – Beállítások és onboarding (Preferences használatával)

Cél: A kulcs–érték alapú adattárolás megismerése a Preferences használatával, valamint annak bemutatása, hogyan őrizhetők meg könnyűsúlyú felhasználói beállítások az alkalmazás újraindítása után is.

Felhasználói történet: Felhasználóként szeretném konfigurálni az alkalmazás beállításait – például a témát, az értesítéseket és a felhasználónevet –, és szeretném, ha az alkalmazás ezeket a munkamenetek között is megjegyezné. Ha még soha nem nyitottam meg az alkalmazást, egy egyszerű onboarding folyamat vezessen végig a kezdeti beállításokon.

Képernyők**1. Onboarding oldal – csak az első indításkor**

- Bemenetek:
 - felhasználónév – szöveg,
 - téma – Light / Dark / System,
 - értesítések engedélyezése – Switch.
- Gomb:
 - Continue – legyen letiltva addig, amíg a felhasználónév mező üres.

2. Beállítások oldal – mindig elérhető

- Ugyanazok a mezők, mint az onboarding oldalon, továbbá:
 - Reset to Defaults,
 - Clear All,
 - Export Settings,
 - Import Settings.

Adatok (Preferences kulcsok)

- prefs.username: string
- prefs.theme: string ∈ {Light, Dark, System}
- prefs.notifications.enabled: bool
- prefs.firstRunCompleted: bool
- prefs.lastOpenedAtUtc: DateTime – minden alkalmazásindításkor frissítendő

Működés

- **Az alkalmazás indításakor:**

- ha a `firstRunCompleted` értéke `false`, vagy a kulcs hiányzik → navigáljon az Onboarding oldalra,
 - egyébként → navigáljon a főoldalra vagy a Beállítások oldalra – szabadon választható –, és frissítse a `lastOpenedAtUtc` értékét.
- A beállításokat mentse minden módosításkor, vagy használjon külön Save gombot.
- **Reset to Defaults:** állítsa be:
 - felhasználónév: "",
 - téma: System,
 - értesítések: false.
- **Clear All:** távolítsa el az alkalmazás összes beállításkulcsát, és jelenítsen meg megerősítést.
- **Export / Import:**
 - **Export:** készítsen egy JSON objektumot a három beállítással – a `lastOpenedAtUtc` ne szerepeljen benne –, majd jelenítse meg egy csak olvasható szerkesztőmezőben, vagy másolja a vágólapra.
 - **Import:** illesszen be JSON szöveget, ellenőrizze a sémát, majd atomi módon frissítse a Preferences értékeket. Érvénytelen adat esetén jelenítsen meg hibaüzenetet.

Szélső esetek

- Üres felhasználónév esetén az onboarding oldalon a Continue gomb legyen letiltva; a Beállítások oldalon az üres érték engedélyezett, de jelenjen meg figyelmeztetés.
- Ha az importált JSON-ból hiányoznak mezők, utasítsa el az importálást konkrét hibaüzenettel.
- A téma módosítása azonnal lépjen életbe, és frissítse az alkalmazás témáját.

Elfogadási kritériumok

- Minden érték megmarad az alkalmazás újraindítása után.
 - Az első indítási onboarding pontosan egyszer jelenik meg, kivéve, ha a Clear All műveletet végrehajtják.
 - Az exportálás, majd importálás ugyanazokat a beállításokat állítja vissza.
 - A témaváltás az alkalmazás újraindítása nélkül frissíti a felhasználói felületet.
-

2. feladat – Személyes jegyzetek fájlokban (lokális fájlok + JSON index)

Cél: Az alkalmazás saját sandbox könyvtárának (FileSystem.AppDataDirectory) használata egyszerű dokumentumok tartós tárolására és kezelésére JSON index és jegyzetenként külön szövegfájl segítségével.

Felhasználói történet: Felhasználóként szeretnék egyszerű szöveges jegyzeteket létrehozni, szerkeszteni, listázni, keresni és törölni. A jegyzetek helyben, offline módon legyenek tárolva.

Képernyők**1. Jegyzetlista oldal**

- Vezérlők:
 - keresőmező cím alapján,
 - New Note,
 - Delete Selected,
 - Export All,
 - Import.
- A listaelemek jelenítsék meg:
 - a címet,
 - az utolsó módosítás időpontját.

2. Jegyzetszerkesztő oldal

- Bemenetek:
 - cím – szöveg,
 - tartalom – többsoros szerkesztő.
- Gombok:
 - Save,
 - Share,
 - Delete.

Tárolási struktúra az AppDataDirectory könyvtáron belül

- notes/index.json – jegyzet-metaadat objektumok tömbje: { id, title, modifiedUtc, fileName }
- notes/<id>.txt – a jegyzet törzsszövege UTF-8 kódolással
- Opcionálisan: notes/.trash/ – a logikailag törölt fájlok tárolására

Működés

- **Létrehozás:** generáljon egy GUID azonosítót, hozza létre a <id>.txt fájlt, majd adja hozzá a bejegyzést az index.json fájlhoz.
- **Módosítás:** írja felül a <id>.txt fájlt, majd frissítse a modifiedUtc értékét az indexben.
- **Törlés:** távolítsa el a bejegyzést az indexből; ezután vagy törölje a fájlt, vagy helyezze át a .trash könyvtárba.
- **Keresés:** kliensoldali szűrés cím alapján, debounce késleltetéssel; ne olvassa be az összes jegyzet törzsszövegét.
- **Export All:** tömörítse ZIP állományba a teljes notes/ könyvtárat notes_export_YYYYMMDD.zip néven az alkalmazás könyvtárán belül, majd jelenítse meg a létrehozott fájl végleges elérési útját.
- **Import:** válasszon ki egy, az alkalmazás által korábban létrehozott .zip fájlt → ellenőrizze annak szerkezetét → egyesítse az aktuális adattárral. Azonosítóütközés esetén generáljon új azonosítókat a duplikált elemekhez, és frissítse az indexet.
- **Megosztás:** ossza meg az aktuális jegyzet törzsszövegét a platform megosztási szolgáltatásával szöveges tartalomként.

Korlátozások

- Az indexnek és a fájloknak konzisztens állapotban kell maradniuk: az indexet csak a fájlműveletek sikeres befejezése után írja ki; fontolja meg ideiglenes fájl és atomi áthelyezés használatát.
- Nagyméretű jegyzetek esetén a listaoldal ne töltse be a fájlok törzsszövegét.
- Külső engedélyekre nincs szükség; használja az alkalmazás saját adatkönyvtárát.

Elfogadási kritériumok

- A jegyzetek megmaradnak az alkalmazás újraindítása után.
 - Létrehozáskor, szerkesztéskor és törléskor az index és a fájlok is frissülnek.
 - Az exportálást követő importálás helyreállítja a jegyzeteket, azonosító-újrakiosztással vagy anélkül.
 - A keresés kis- és nagybetűtől független, azonnal működik, és kizárólag a cím mezőt vizsgálja.
-

3. feladat – Fényképes napló (lokális fájlok: képek + JSON metaadatok + bélyegképek)

Cél: A bináris fájlok tartós tárolásának, a metaadatok indexelésének és a teljesítmény javítására szolgáló bélyegkép-generálásnak a gyakorlása.

Felhasználói történet: Felhasználóként szeretnék fényképet készíteni vagy kiválasztani, képaláírást és címkéket hozzáadni, majd a képeket bélyegképek segítségével gyorsan böngészni.

Képernyők**1. Galéria oldal**

- Bélyegképekből álló rács 3–4 oszloppal
- Add Photo
- keresés címkék és képaláírás alapján

2. Fénykép részletei oldal

- teljes méretű kép
- szerkeszthető képaláírás
- vesszővel elválasztott címkék
- Save
- Delete
- Share

Tárolási struktúra

- photos/index.json – objektumok tömbje: { id, fileName, thumbFileName, caption, tags[], createdUtc }
- photos/<id>.jpg – eredeti kép, amely a kamerából vagy a képválasztóból kerül átmásolásra
- photos/thumbs/<id>.jpg – átméretezett bélyegkép, például legfeljebb 256 px-es maximális mérettel, a gyors listamegjelenítés érdekében

Működés

- **Fénykép hozzáadása:** a felhasználó választhasson kamera vagy galéria között; másolja a forrásképet a <id>.jpg fájlba; generáljon bélyegképet; frissítse az indexet.
- **Metaadatok szerkesztése:** frissítse a képaláírást és a címkéket az indexben; az eredeti képfájlt ne módosítsa.

- **Törlés:** távolítsa el a bejegyzést az indexből, és törölje mindkét fájlt.
- **Keresés:** szűrés címkék és képaláírás alapján, kis- és nagybetűtől független részszöveg-illesztéssel.
- **Megosztás:** ossza meg az eredeti képfájlt a rendszer megosztási felületén keresztül.
- **Teljesítmény:** a Galéria oldal kizárólag bélyegképeket jelenítsen meg; a listaoldalon soha ne dekódolja az eredeti képeket.

Korlátozások

- **Engedélykezelés:** a kamera és a galéria elérésekor az első használat során jelenjenek meg a platform engedélykérő ablakai.
- **Atomi frissítés:** az indexet először ideiglenes fájlba írja ki, majd helyezze át vagy cserélje le az eredeti fájlt a sérülés elkerülése érdekében.
- Ha egy bélyegkép hiányzik – például importálás vagy visszaállítás után –, generálja újra.

Elfogadási kritériumok

- Nagyméretű galériák esetén is folyamatos marad a felület működése a bélyegképek használata miatt.
- A képaláírások és címkék megfelelően kiírhatók és visszaolvashatók az index.json fájlból.
- Fénykép törlésekor a hozzá tartozó fájlok is törlődnek.
- Külső forrásból származó fájlok importálása opcionális, de ha megvalósítja, új azonosítókat kell kiosztani, és bélyegképeket kell generálni.

4. feladat – Feladatkezelő EF Core használatával (SQLite adatbázis, Entity Framework Core)

Cél: Az Entity Framework Core és az SQLite provider megismerése strukturált adatok, kapcsolatok és lekérdezések kezelésére.

Felhasználói történet: Felhasználóként szeretnék projekteket és feladatokat létrehozni, feladatokat késznek jelölni, valamint szűrni és rendezni őket. Minden adat lokálisan, SQLite adatbázisban legyen tartósan tárolva.

Adatmodell – entitások, kód nélkül

- Project: Id (int, PK), Name (string, kötelező, egyedi), CreatedUtc (DateTime)
- TaskItem: Id (int, PK), ProjectId (FK → Project), Title (string, kötelező), IsDone (bool), Priority (int, 1..5), DueUtc (DateTime?), CreatedUtc (DateTime)

Képernyők

1. Projektek oldal: projektek listázása; Add Project, Delete, navigáció a projekt feladataihoz.

2. Projekt feladatai oldal: a kiválasztott projekthez tartozó feladatok megjelenítése; Add Task, kész állapot váltása, Delete, szűrők (All / Active / Completed), rendezés határidő vagy prioritás szerint.

3. Statisztika oldal – opcionális: projektenkénti darabszámok, lejárt feladatok, teljesítési arány megjelenítése.

EF Core követelmények

- Használja az EF Core SQLite providert saját DbContext-tel.
- Konfiguráljon migrációkat – modell snapshot és sémafrissítések.
- Definiálja a kapcsolatot: Project 1..* TaskItem.
- Hozzon létre indexet a TaskItem(ProjectId) mezőre, opcionálisan a TaskItem(IsDone) mezőre is a szűrésekhez.
- **Kezdeti adatok:** az első indításkor hozzon létre egy mintaprojektet néhány feladattal.
- Ahol rendelkezésre állnak, használja az adatbázis-műveletek aszinkron API-jait.

Működés

- Projekt létrehozásakor és törlésekor a hozzá tartozó feladatokat megfelelően kezelje a kaszkádolt törlés; biztosítsa a referenciális integritást.
- A szűrés és rendezés adatbázis-szinten, lekérdezések segítségével történjen, ne memóriában.
- Az IsDone érték módosítása után a felhasználói felület azonnal frissüljön.
- Opcionálisan jelenítse meg a lejárt feladatokat: DueUtc < now && !IsDone.

Korlátozások

- Használjon DI-t a DbContext regisztrálásához – scoped vagy alkalmazásonként singleton –, de biztosítsa a biztonságos használatát a UI szálon.
- Az adatbázis fájlja a FileSystem.AppDataDirectory könyvtár alatt helyezkedjen el.
- Migrációs hiba esetén ajánlja fel az adatbázis teljes törlését és újraépítését; az adatbázis törlése előtt kérjen megerősítést a felhasználótól.

Elfogadási kritériumok

- A sémamigráció sikeresen lefut az alkalmazás különböző verziói között.
- A szűrési és rendezési lekérdezések hatékonyan működnek; a kód ne végezzen szükségtelen teljes tábla-beolvasást.
- A projektek és feladatok megmaradnak az alkalmazás újraindítása után.
- Projekt törlésekor a hozzá tartozó feladatok kaszkádolt módon törlődnek.

5. feladat – Személyes pénzügyi nyilvántartás (EF Core + Preferences + lokális fájlimport/import)

Cél: A három megoldás együttes alkalmazása: EF Core – SQLite – a tranzakciókhoz, Preferences a felhasználói beállításokhoz, valamint lokális fájlok CSV/JSON exportáláshoz és importáláshoz – biztonsági mentéshez és visszaállításhoz. Opcionálisan használjon SecureStorage-ot titkosítási kulcs tárolására, ha a biztonsági mentéseket titkosítani szeretné.

Felhasználói történet: Felhasználóként szeretném kategóriák szerint rögzíteni a bevételeimet és kiadásaimat, dátumtartomány alapján szűrni, összesítéseket megjeleníteni, módosítani a pénznem- és területi beállításokat, valamint exportálni és importálni az adataimat.

Adatmodell – entitások

- Category: Id (int, PK), Name (string, kötelező, egyedi), Type (enum: Income / Expense)
- Transaction: Id (int, PK), CategoryId (FK → Category), Amount (decimal), OccurredOn (DateOnly vagy DateTime), Note (string?), CreatedUtc (DateTime)
- Indexek:
 - Transaction(OccurredOn)
 - Transaction(CategoryId)

Preferences kulcsok

- prefs.currency: string – ISO kód, például EUR
- prefs.locale: string – például en-US
- prefs.firstRunCompleted: bool – első indításkor alapértelmezett kategóriák létrehozásához

Képernyők**1. Dashboard oldal**

- dátumtartomány-választó:
 - This Month
 - Last Month
 - Custom
- összesítések:
 - bevétel,
 - kiadás,
 - egyenleg,
- egyszerű diagram – opcionális.

2. Tranzakciók oldal

- lista szűrőkkel:
 - dátumtartomány,
 - kategória,
 - típus,
- Add
- Edit
- Delete

3. Kategóriák oldal

- kategóriák kezelése,
- akadályozza meg egy hivatkozott kategória törlését, vagy kérje a hozzá tartozó elemek átsorolását.

4. Beállítások oldal

- pénznem,
- locale,
- Export,
- Import,
- Reset.

Tárolás és fájlműveletek

- **EF Core SQLite:** minden kategória és tranzakció lokálisan, az AppDataDirectory könyvtár alatt legyen tárolva.

- **Export:** készítsen backup_YYYYMMDD.json fájlt – vagy .csv fájlt a tranzakciókhoz – az AppDataDirectory könyvtárban.
 - A JSON tartalmazza:
 - a séma verzióját,
 - a Preferences részhalmazát – pénznem és locale,
 - a kategóriákat,
 - a tranzakciókat.
- **Import:** válasszon ki egy biztonsági mentési fájlt; ellenőrizze a séma verzióját és a duplikációkat; hajtson végre upsert műveletet a kategóriákon és tranzakciókon.
 - Azonosítóütközés esetén generáljon új azonosítókat az importált elemekhez, és képezze át az idegen kulcsokat.
- **Opcionális SecureStorage:** tároljon titkosítási jelszót vagy kulcsot; ha rendelkezésre áll, titkosítsa és dekódolja a biztonsági mentési fájlokat.

Működés

- Minden lekérdezés adatbázis-szintű szűrést használjon, például dátumtartomány és kategória alapján.
- A pénznem vagy locale módosítása frissítse a formázást az egész alkalmazásban – converter vagy culture használatával.
- Első indításkor hozzon létre példakategóriákat, például:
 - Salary
 - Rent
 - Food
 - stb.
- A véletlenül végrehajtott destruktív műveleteket megerősítő kérdésekkel akadályozza meg.

Korlátozások

- Importálás közben használjon tranzakciót: EF DbContext.Database.BeginTransaction, hogy biztosítsa az atomi működést.
- A UI szálon minden adatbázis-műveletet aszinkron módon hajtson végre.
- Nagyméretű adathalmazok esetén biztosítson megfelelő kezelést, például lapozással vagy inkrementális betöltéssel.

Elfogadási kritériumok

- Export → adatbázis törlése → Import után az adathalmaz álljon helyre. Az azonosítók eltérhetnek, de a kapcsolatoknak és az összesítéseknek egyezniük kell.
 - A Dashboard összesítései pontosan tükrözzék az alkalmazott szűrőket.
 - A Preferences-ben tárolt pénznem- és locale-beállítások maradjanak meg, és megfelelően alkalmazódjanak a megjelenített összegekre.
-

Általános útmutató minden feladathoz

- **MVVM:** az adattárolási logikát szolgáltatások vagy repository-k mögött valósítsa meg; a ViewModel-ek koordinálják a működést; a View-k kizárólag bindingot használnak.
- **DI:** a szolgáltatásokat és a DbContext-et a MauiProgram osztályban regisztrálja; az oldalak konstruktorinjektálással kapják meg a ViewModel-eket. Nem DI-alapú demonstrációk esetén a BindingContext beállítható a code-behind fájlban.
- **AppDataDirectory:** az abszolút fájlvonalakat minden esetben a FileSystem.AppDataDirectory értékéből kiindulva állítsa elő.
- **Atomi írás JSON indexfájlok esetén:** először ideiglenes fájlba írjon, majd cserélje le az eredeti fájlt.
- **Validáció:** hiba esetén a feldolgozás minél korábban álljon le, és jelenjen meg egyértelmű hibaüzenet, például hibás formátumú import esetén.
- **Tesztelés:** indítsa újra az alkalmazást a tartós adattárolás ellenőrzéséhez; teszteljen sérült fájlokkal; próbáljon ki nagyméretű listákat; ellenőrizze a keresési és szűrési funkciók szélső eseteit.